Learning Sequence Models

Counts, smoothing, evaluation, and generation

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

Estimate the transition matrix from data

An observed sequence

Suppose the two states A and B produce

A A B A B B B A

There are seven observed transitions:

$A \to A, A \to B, B \to A, A \to B,$

$B \to B, B \to B, B \to A.$

Count each ordered pair.

Transition count table

current	next A	next B	row total
A	1	2	3
B	2	2	4

Rows correspond to current states; columns correspond to next states, matching L25.

Normalize each row

For current state A:

$$\hat{P}_{AA} = \frac{1}{3}, \quad \hat{P}_{AB} = \frac{2}{3}.$$

For current state B:

$$\hat{P}_{BA} = \frac{2}{4} = \frac{1}{2}, \quad \hat{P}_{BB} = \frac{2}{4} = \frac{1}{2}.$$

Thus

$$\hat{P} = \begin{pmatrix} \frac{1}{3} & \frac{2}{3} \\ \frac{1}{2} & \frac{1}{2} \end{pmatrix}.$$

Why row normalization is plausible

Given the current state i, the next state must be one of the columns j.

The empirical conditional frequency is

$$\frac{\text{number of observed } i \to j \text{ transitions}}{\text{number of transitions leaving } i}.$$

The calculation is also the maximum-likelihood estimator. We now derive it.

Learning outcomes

By the end of this lecture you will be able to:

1. estimate a transition matrix from counts,
2. derive ⭐ $\hat{P}_{ij} = \frac{n_{ij}}{n_{i\cdot}}$ using Lagrange multipliers,
3. apply Dirichlet smoothing and distinguish prediction from MAP,
4. build a character n-gram model,
5. score held-out text in bits per character and perplexity,
6. sample new sequences from a fitted model,
7. identify how every course module appears in this one artifact,
8. and locate the next mathematical steps toward ML and DL.

Maximum likelihood for one row

Likelihood of transition counts

Let n_{ij} count transitions from state i to state j.

Ignoring the initial-state term, the path likelihood is

$$L(P) = \prod_i \prod_j P_{ij}^{n_{ij}}.$$

The log-likelihood is

$$\ell(P) = \sum_i \sum_j n_{ij} \log P_{ij}.$$

Constraints live row by row

Every row is a probability distribution:

$$\sum_j P_{ij} = 1, \quad P_{ij} \geq 0.$$

The log-likelihood separates across rows. For one fixed current state i, solve

$$\max_{P_{i1},\ldots,P_{iK}} \quad \sum_j n_{ij} \log P_{ij}$$

$$\text{subject to} \quad \sum_j P_{ij} = 1.$$

⭐ Form the row Lagrangian

D DERIVATION

$$\mathcal{L}_i = \sum_j n_{ij} \log P_{ij} - \lambda_{i\left(\sum_j P_{ij} - 1\right)}.$$

First assume every $n_{ij} > 0$, so the maximizing row is in the simplex interior.

Differentiate with respect to P_{ij}:

$$\partial \frac{\mathcal{L}_i}{\partial P_{ij}} = \frac{n_{ij}}{P_{ij}} - \lambda_i = 0.$$

Therefore

$$P_{ij} = \frac{n_{ij}}{\lambda_i}.$$

⭐ Use the row-sum constraint

D DERIVATION

$$\sum_j P_{ij} = \sum_j \frac{n_{ij}}{\lambda_i} = 1.$$

Define the row total

$$n_{i\cdot} = \sum_j n_{ij}.$$

Then

$$\lambda_i = n_{i\cdot}.$$

⭐ The transition MLE

D DERIVATION

Substitute $\lambda_i = n_{i\cdot}$:

$$\hat{P}_{ij} = \frac{n_{ij}}{n_{i\cdot}} \quad \text{for} \quad n_{i\cdot} > 0.$$

The maximum-likelihood transition probability is the empirical fraction of departures from i that arrive at j.

If $n_{ij} = 0$, KKT places that coordinate on the boundary at $\hat{P}_{ij} = 0$, giving the same formula. If $n_{i\cdot} = 0$, the row was never observed and its transition probabilities are not identifiable without a prior or another modeling choice.

Return to the A/B sequence

$$n_{A\cdot} = 3, \quad n_{B\cdot} = 4.$$

$$\hat{P}_{AA} = \frac{1}{3}, \quad \hat{P}_{AB} = \frac{2}{3},$$

$$\hat{P}_{BA} = \frac{2}{4}, \quad \hat{P}_{BB} = \frac{2}{4}.$$

The intuitive row-normalization calculation and the constrained MLE agree.

The initial distribution

The full sequence likelihood also contains $p(X_1)$.

Possible choices:

1. estimate an initial-state distribution from many independent sequences,
2. fix the observed first state,
3. initialize from a known external distribution,
4. use the stationary distribution.

Transition estimation and initialization answer different questions.

Code from a sequence

```
import numpy as np

path = np.array([0, 0, 1, 0, 1, 1, 1, 0])
counts = np.zeros((2, 2), dtype=int)

for current, nxt in zip(path[:-1], path[1:]):
    counts[current, nxt] += 1

P = counts / counts.sum(axis=1, keepdims=True)
print(counts)  # [[1 2], [2 2]]
print(P)       # [[.333 .667], [.5 .5]]
```

Question: estimate a row

Q QUESTION

From state C, observed next-state counts are $(6, 3, 1)$. What is the MLE row?

A. $(6, 3, 1)$

B. $(0.6, 0.3, 0.1)$

C. $\left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$

D. $\left(\frac{7}{13}, \frac{4}{13}, \frac{2}{13}\right)$

Correct: B — $(0.6, 0.3, 0.1)$

Why: Divide each count by the row total $6 + 3 + 1 = 10$.

A bigram language model

Characters are states

Let each character be a state: space plus `a` through `z`.

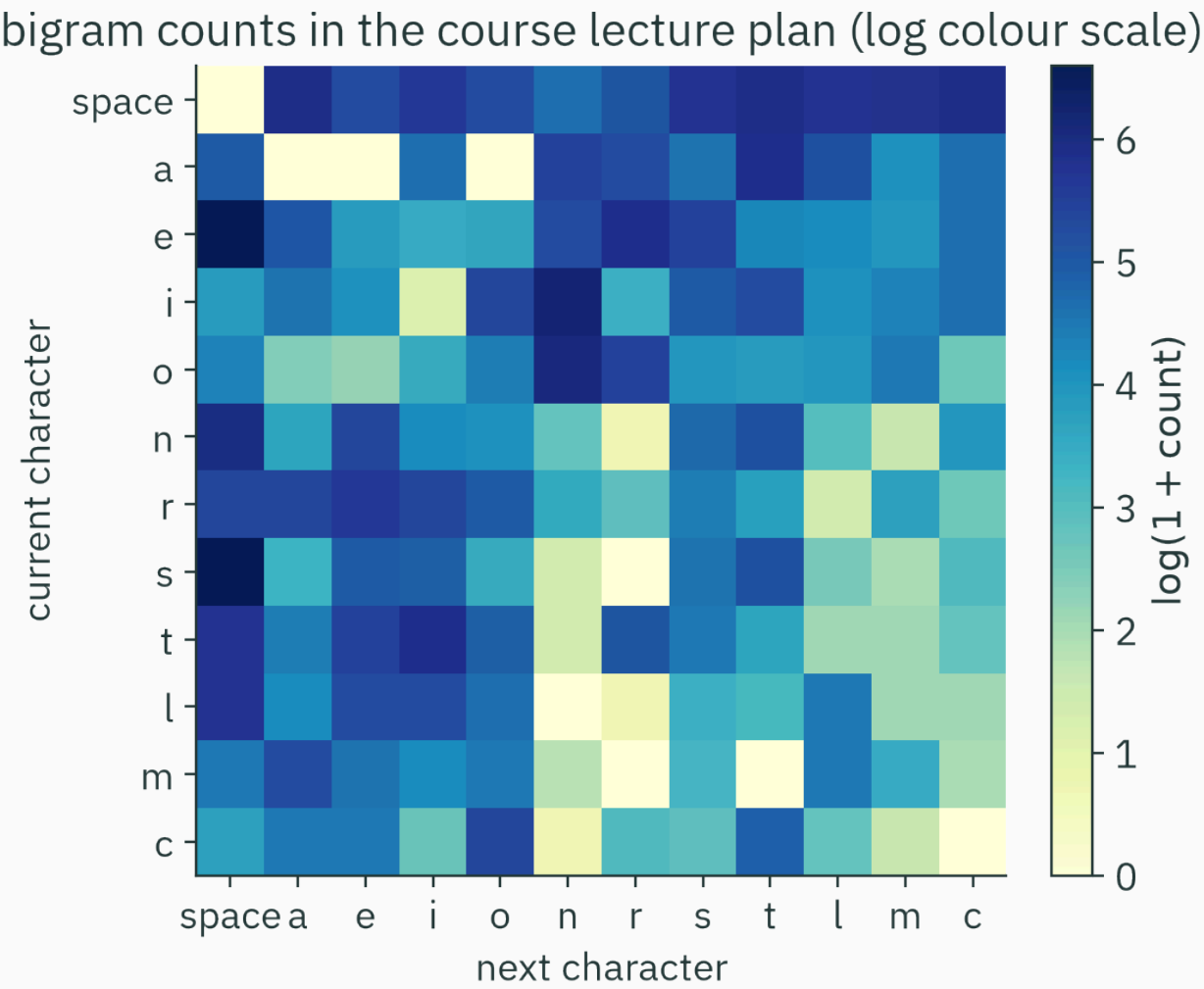
A character bigram model assumes

$$p(x_t|x_1, ..., x_{t-1}) \approx p(x_t|x_{t-1}).$$

The transition matrix has 27×27 entries. Row `t` stores probabilities for the character following `t`.

Counts from the course plan

V VISUAL



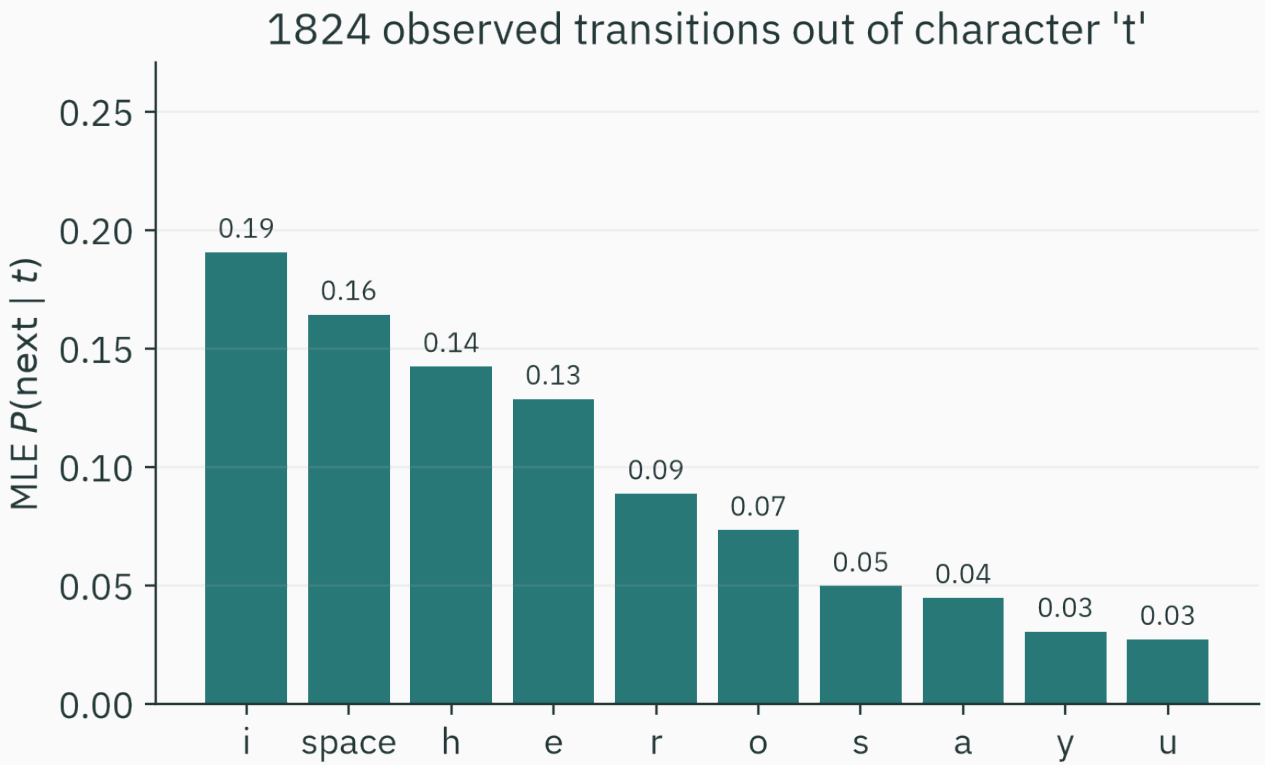
Counts from the course plan

V VISUAL

The heatmap shows a subset of the alphabet. Common spelling and spacing patterns appear as large counts.

One learned row

V VISUAL



The model learns from data that `t` is often followed by a space, `h`, or another common continuation.

Build character counts

```
from collections import defaultdict, Counter

text = clean(open("lecture-plan-detailed.md").read())
counts = defaultdict(Counter)

for current, nxt in zip(text[:-1], text[1:]):
    counts[current][nxt] += 1

row_t = counts["t"]
total_t = sum(row_t.values())
p_t = {ch: n/total_t for ch, n in row_t.items()}
```

Bigram likelihood

For text $x_1, ..., x_T$,

$$p(x_1, ..., x_T) = p(x_1) \prod_{t=2}^{T} P_{x_{t-1}, x_t}.$$

The training MLE simply counts every adjacent pair and normalizes rows.

No gradient descent is needed because the constrained optimum has a closed form.

Higher-order n-grams

1. unigram: $p(x_t)$ — no context,
2. bigram: $p(x_t|x_{t-1})$ — one previous character,
3. trigram: $p(x_t|x_{t-2}, x_{t-1})$ — two previous characters,
4. 4-gram: three previous characters.

A higher-order model is still first-order Markov after treating the whole context as the state.

Parameter growth

With alphabet size K and context length m, a dense table contains

$$K^{m(K-1)}$$

free transition parameters.

For $K = 27$:

1. bigram ($m = 1$): 702 free parameters,
2. trigram ($m = 2$): $18,954$,
3. 4-gram ($m = 3$): $511,758$.

Longer context needs much more data.

Unseen does not mean impossible

The zero-probability problem

Suppose one row has counts

$(8, 2, 0, 0)$.

MLE gives

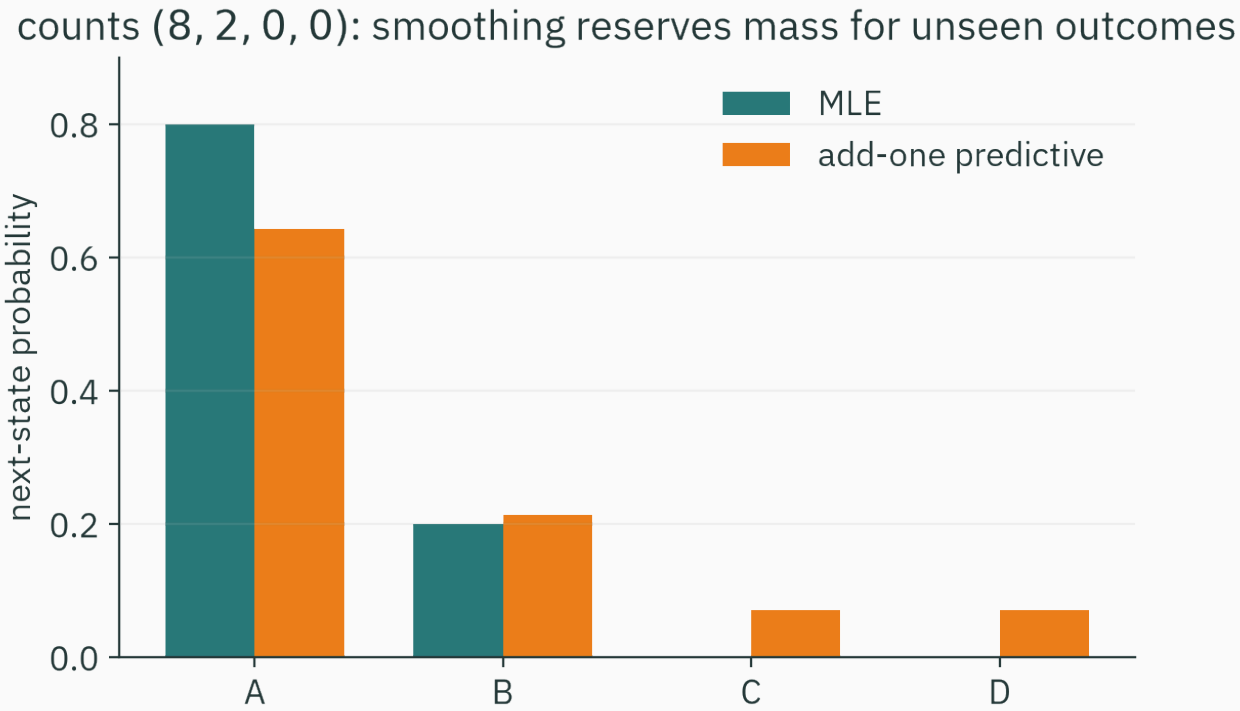
$(0.8, 0.2, 0, 0)$.

If either unseen transition occurs in test data, its log loss is infinite:

$-\log 0 = \infty$.

Add-one smoothing

V VISUAL



Add-one prediction changes $(8, 2, 0, 0)$ to

Add-one smoothing

V VISUAL

$$\left(\frac{9}{14}, \frac{3}{14}, \frac{1}{14}, \frac{1}{14}\right).$$

Dirichlet prior

For a transition row $\boldsymbol{P}_i$, choose

$$\boldsymbol{P}_i \sim \text{Dirichlet}(\alpha_1, ..., \alpha_K).$$

After counts $n_{i1}, ..., n_{iK}$, conjugacy gives

$$\boldsymbol{P}_i | \text{data} \sim \text{Dirichlet}(n_{i1} + \alpha_1, ..., n_{iK} + \alpha_K).$$

This is the multinomial analogue of L15′s Beta–Bernoulli update.

Posterior predictive probabilities

The probability of the next transition, averaged over the posterior, is

$$P(X_{\text{next}} = j | X_{\text{current}} = i, \text{data}) = \frac{n_{ij} + \alpha_j}{n_{i\cdot} + \sum_k \alpha_k}.$$

For symmetric $\alpha_j = \alpha$, this is add-α smoothing.

Predictive mean is not always MAP

For $\alpha_j > 1$, the row-wise MAP is

$$\frac{n_{ij} + \alpha_j - 1}{n_{i\cdot} + \sum_k \alpha_k - K}.$$

The posterior predictive mean uses $n_{ij} + \alpha_j$ instead.

“Add-one smoothing” usually refers to predictive probabilities under a uniform Dirichlet prior, not the MAP for that prior.

Choose the smoothing strength

Large α pulls rows strongly toward the prior mean.

Small α stays closer to counts but may assign extremely small probabilities to unseen transitions.

Choose α on validation data using held-out log loss, not training likelihood alone.

Test on text not used for counting

Held-out cross-entropy

For a test sequence of length T, bits per character is

$$\mathrm{BPC} = -\frac{1}{T-m}\sum_{t=m+1}^{T} \log_2 q(x_t | x_{t-m}, \ldots, x_{t-1}).$$

Lower BPC means the model assigns higher probability to the observed continuation.

Character-level perplexity is

2^{BPC}.

Train/test split

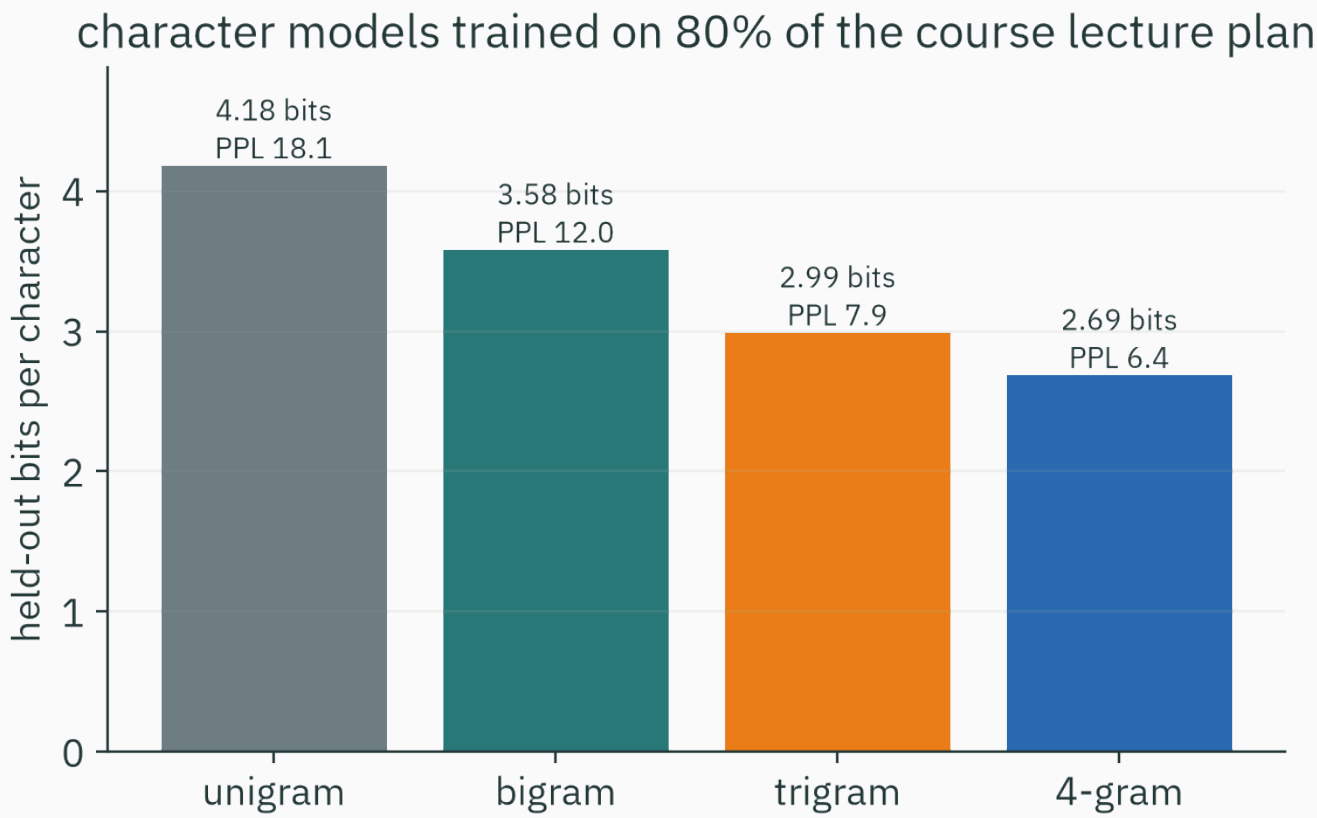
For the course lecture plan:

1. clean to lowercase letters and spaces,
2. first 80%: estimate counts,
3. final 20%: evaluate,
4. use add-0.1 smoothing,
5. never update counts on the test segment.

Chronological splitting better reflects sequence prediction than random character shuffling.

Context reduces held-out coding cost

V VISUAL



Held-out BPC falls from 4.18 for a unigram to 2.69 for a 4-gram on this corpus.

Convert BPC to effective choices

model	BPC	perplexity
unigram	4.181	18.1
bigram	3.581	12.0
trigram	2.991	7.9
4-gram	2.687	6.4

The longer context narrows the effective next-character choice set.

More context is not automatically better

A higher-order model can have lower training loss but worse test loss because many contexts are rare.

Control the trade-off with:

1. smoothing,
2. backoff or interpolation with shorter contexts,
3. validation data,
4. parameter sharing.

Neural language models replace explicit tables with learned shared representations.

Question: compare models fairly

Q QUESTION

Which comparison makes character-level BPC meaningful?

A. different test corpora

B. same corpus and same character alphabet

C. training loss for one model and test loss for another

D. different log bases without conversion

Answer

A ANSWER

Correct: B — same corpus and same character alphabet

Why: Cross-entropy is meaningful only when the evaluated data and unit of a symbol are held fixed.

Sample the fitted model

Generation is repeated conditional sampling

Given current context c_t:

1. retrieve the smoothed row $q(\cdot \mid c_t)$,
2. sample the next character x_{t+1},
3. append it to the output,
4. update the context,
5. repeat.

The probabilities used for scoring are the same probabilities used for generation.

Bigram sampler

```
rng = np.random.default_rng(8)
context = "t"
output = [context]

for _ in range(300):
    probs = smoothed_row(context)
    nxt = rng.choice(alphabet, p=probs)
    output.append(nxt)
    context = nxt

print("".join(output))
```

What a bigram learns

Sample from the course-plan bigram model:

```
fulothecanaie s denivala chenen ch gers vex herde inghar
ne d heg pe tion ceadeximaty de itangsinim sata lly bin...
```

Local letter pairs look plausible; words and sentences do not. One previous character is insufficient for language structure.

A 4-gram sample

```
the exchand compostep newton derivation lives x ent inputed
bound float does ch dernoully ipynb invert convectors manythis...
```

Short fragments from the course vocabulary appear, but long-range syntax remains weak.

Swap in a larger corpus

The same notebook can replace the course plan with Tiny Shakespeare:

```
text = open("input.txt", encoding="utf-8").read()
model = fit_ngram(text, order=3, alpha=0.1)
print(sample(model, n=1000, seed=0))
```

The algorithm does not change. Corpus size, alphabet, and context order change the fitted probabilities.

Sampling temperature

★ OPTIONAL

Given probabilities q_j, temperature $\tau > 0$ defines

$$q_j^\tau \propto \exp\left(\frac{\log q_j}{\tau}\right).$$

1. $\tau < 1$: sharper, more repetitive samples,
2. $\tau = 1$: original model,
3. $\tau > 1$: flatter, more varied, more errors.

Temperature changes generation; it does not improve held-out likelihood.

Every module appears here

Machine numbers and linear algebra

lecture	object	use in the sequence model
L2	floating point	sum log probabilities; never multiply thousands of tiny terms
L3	vectors	a state distribution is a probability vector
L4	matrices	$\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t P$
L5	eigenvectors	stationary distribution has eigenvalue one
L6	low rank	large transition tables may admit compressed structure

Calculus and automatic differentiation

lecture	object	use beyond the count model
L7	local linearization	optimize parameterized sequence models
L8	gradients	direction of cross-entropy change
L9	Jacobians/Hessians	sensitivity and curvature of model outputs
L10	finite/forward diff	check small derivatives
L11	reverse mode	backpropagate sequence loss through shared parameters

Probability and estimation

lecture	object	use in the sequence model
L12–13	distributions	represent uncertainty and dependence
L14	MLE	row-normalized transition counts
L15	MAP / conjugacy	Dirichlet smoothing of each row

Counting, normalization, and smoothing are estimation procedures—not ad hoc text tricks.

Optimization

lecture	object	use in the sequence model
L16	convexity	row log-likelihood has one global optimum
L17–18	first/second order	fit models without a count closed form
L19	Lagrange multipliers	derive row-normalized MLE
L20	KKT	handle probability non-negativity and active zeros
L21	structured solvers	recognize special training objectives

Information and sequences

lecture	object	use in the sequence model
L22	entropy	irreducible uncertainty of a source
L23	coding	probability becomes description length
L24	cross-entropy / KL	held-out BPC and model mismatch
L25	Markov chains	transition matrix, stationary distribution, PageRank
L26	learning sequences	estimate, smooth, score, and sample

From n-grams to neural language models

An n-gram model stores a separate probability row for every discrete context.

A neural language model:

1. maps tokens and contexts to vectors,
2. shares parameters across related contexts,
3. produces logits and softmax probabilities,
4. minimizes the same cross-entropy,
5. samples from the same conditional factorization.

The probability and loss framework remains; the parameterization becomes much richer.

What you can now build and check

A compact sequence-model workflow

1. Define states or symbols and context length.
2. Split training, validation, and test sequences.
3. Count transitions or choose a parameterized model.
4. Normalize with appropriate smoothing.
5. Evaluate held-out cross-entropy in a clear unit.
6. Compare against simpler baselines.
7. Inspect generated samples and failure modes.
8. Record assumptions about stationarity and memory.

Summary

Transition-count MLE is row normalization; smoothing makes prediction finite; cross-entropy evaluates the fitted sequence model.

1. For an observed row, $\hat{P}_{ij} = \frac{n_{ij}}{n_{i\cdot}}$ follows from constrained maximum likelihood.
2. A Dirichlet prior yields add-α posterior predictive probabilities.
3. Longer context can reduce BPC but increases parameter and data requirements.
4. Generation repeatedly samples the same conditional probabilities used for scoring.
5. The character model uses every mathematical module in the course.

Practice

1. Estimate P from transitions $A \to A$ twice, $A \to B$ once, $B \to A$ three times, and $B \to B$ once.
2. Apply add-one smoothing to the A row.
3. A test sequence receives probabilities $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}\right)$. Find total bits and average BPC.
4. Why can a 10-gram have worse held-out BPC than a 4-gram?

Practice answers

A ANSWER

1. $\hat{P} = \begin{pmatrix} \frac{2}{3} & \frac{1}{3} \\ \frac{3}{4} & \frac{1}{4} \end{pmatrix}$.
2. Counts $(2, 1)$ become predictive probabilities $\left(\frac{3}{5}, \frac{2}{5}\right)$.
3. Total $= 1 + 2 + 3 = 6$ bits; average $= 2$ BPC.
4. Most long contexts are rare or unseen, so their estimates have high variance unless data sharing, smoothing, or backoff is strong enough.

What follows this course

In a machine-learning course, these foundations support:

1. linear and logistic regression,
2. regularization and model selection,
3. support vector machines and kernels,
4. tree ensembles and probabilistic models.

In a deep-learning course, they support:

1. neural networks and backpropagation,
2. embeddings and attention,
3. convolutional and sequence models,
4. generative models and large language models.

Final checklist

You can now ask of an AI method:

1. What is represented as a vector or matrix?
2. Which probability model produced the loss?
3. Which derivative is being computed, and in what shape?
4. What local approximation justifies the update?
5. Which constraint or numerical scale can break it?
6. What does its cross-entropy say in bits?
7. Which assumptions connect observations across time?

You have fitted, evaluated, and sampled a language model; larger models change the parameterization, not the mathematical questions.